TECHNICAL DESIGN DOCUMENT

构建详情状态小结技术方案

BUILD DETAIL STATUS SUMMARY — DESIGN & IMPLEMENTATION

在同一条构建详情接口上，额外返回一张小结卡片，
用一句话说清「现在怎么样、为什么、影响了谁」。

需求编号
#13477

实现 PR
#13705 · issue_13477_detail_reason_ing

适用接口
getBuildRecordByExecuteCount
（ModelRecord，不新增接口）

文档版本
v1.3（图表校正 · 内容对齐版）

蓝盾团队 · CI/CD 技术中台 | 蓝盾 DevOps 平台 bk-ci

CONTENTS

目录

全文 13 章 + 1 附录，共 10 张原理图、25 张对照表

一、背景与目标 03

二、核心设计思路 04

2.1 四条设计原则 04

2.2 整体数据流转架构 05

三、卡片决策逻辑 06

3.1 页面状态与返回卡片对照表 07

四、模块分工与协作 08

4.1 各模块职责 09

五、终态写入机制 10

5.1 写入时序 10

5.2 全场景写入对照表 11

5.3 几个容易踩坑的细节 13

六、失败类型判定规则 14

6.1 判定举例 15

七、读取与动态合成 16

7.1 读取时的关键规则 17

八、阶段准入特殊处理 18

8.1 关键规则 19

九、运行态实时计算 20

9.1 排队计算规则 21

9.2 执行中计算规则 21

十、推送机制与空窗说明 22

10.1 推送时机 22

10.2 三段已知空窗 23

十一、数据结构与前端对接 24

11.1 终态详情 buildEndInfo 24

11.2 endType 枚举全集 25

11.3 运行态详情 buildRunningInfo 26

十二、明确不改的行为 27

十三、上线与验证 28

附录 A　代码与常量索引 29

附录 A（续）读取侧与单测索引 30

怎么读这份文档。第 2～3 章是「为什么这么设计」，先看它就能看懂后面所有细节；第 5～9 章是写入与读取的实现规则，按链路顺序排列；第 11 章是给前端的字段契约；第 12 章是「不要动」的边界，最容易被后人误改，建议重点看。

图表清单

图 1 状态小结卡片的三张典型形态（p03）· 图 2 整体数据流转架构（p05）· 图 3 卡片决策流程（p06）· 图 4 模块分工架构（p08）· 图 5 终态写入时序（p10）· 图 6 失败类型判定流程（p14）· 图 7 读取决策流程（p16）· 图 8 阶段准入时序（p18）· 图 9 运行态计算流程（p20）· 图 10 三次推送时机（p22）

01 BACKGROUND

一、背景与目标

为什么要做这件事 · 要解决什么问题

1.1 现状痛点：只给结论、不给原因

构建详情页原来只有一个孤零零的状态词 —— 「失败」「已取消」「运行中」。用户看到这个词之后， 还得自己去翻编排图、逐个点开 Job 和插件，才能搞清楚三件事：

用户的疑问	现状	代价
是谁取消的?	状态一律显示「已取消」，分不清用户手动取消、系统取消，还是父流水线连带取消	翻取消人、翻父流水线，跨页面跳转
哪一个插件失败了?	只知道构建失败，不知道是执行失败、质量红线、人工审核驳回，还是子流水线挂了	逐个 Job 展开，找红色图标
现在卡在哪?	「运行中」背后可能是排队、等构建机、等人工审核，页面完全不区分	只能干等，或去别处查队列

1.2 方案目标：多给一张会说话的卡片

在同一条详情接口 `getBuildRecordByExecuteCount` 上，额外返回一张状态小结卡片： **还没结束时**说清现在卡在哪一步、前面有谁在排队、有没有需要人介入的环节；**已经结束时**说清为什么结束 （取消 / 失败 / 超时 / 成功）、原因是什么、影响了哪些 Stage / Job / 插件。前端完全按后端给出的大类（ `endCategory` / `runningCategory` ）和已翻译文案渲染， **不自己猜成因。**

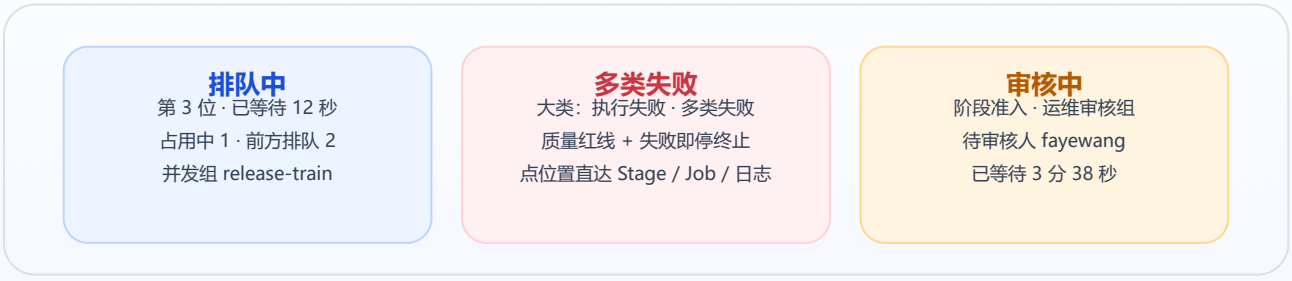


图 1 状态小结卡片的三张典型形态：排队等位 / 失败溯源 / 等待人工，都用同一张卡片承载

1.3 上线结论

可以上线。终态大类与页面状态对齐，FastKill、阶段准入和取消人的口径与实现一致。第 10 节的三段展示窗口是 **推送时序导致的正常空窗**，不是状态写错。注意 `biz-base` 依赖 jOOQ 连库生成，需在 CI 再编一次。

02 PRINCIPLE

二、核心设计思路

四条原则 · 后面所有的流程图和模块分工，都从这四条推导出来

2.1 四条设计原则

原则 1

一张卡片，两种寿命

结束原因确定后**不再变**，写入记录的 `modelVar["buildEndInfo"]`；排队和执行中的等待**每秒都在变**，不落库，打开详情时现算 `buildRunningInfo`。

同一时刻只返回其中一张，避免两张卡片叠在一起。

原则 2

构建级大类跟着页面状态走

卡片大类必须和页面左上角状态是同一类：取消对取消、失败对失败、排队超时对超时、成功对成功。

Job 超时、步骤超时、构建机失联发生在**某个位置上**，构建本身往往以失败或取消收尾，所以构建级卡片**不叫「超时」**，超时分钟数写在原因和那一行位置上。

原则 3

能提前记的成因提前记，但运行中不展示

用户一点取消，取消人、在途插件就固定了；Job 超时、构建机失联也是在当时才知道影响了哪个容器。这些先写入记录。

构建还在跑时，**读取直接不返回这张卡片**；等构建真正结束后，再按最终状态决定保留、覆盖或清掉。

原则 4

阶段准入把「记录」和「历史」拆开看

审核挂起时，记录和历史都是 `STAGE_SUCCESS`，卡片是「审核中」。最后一组点同意后，记录先变运行，历史表要等事务提交才变运行。

运行态看的是历史表，所以要等历史提交后再推一次详情。驳回时记录**必须保持 CANCELED**，`finally` 才能在「已取消」前提下跑完收尾阶段。

一句话记住这四条。终态是「一次写、永不变」的结论，运行态是「每次读、现算」的快照；构建级只说大类、位置级才说细节；成因要在现场记下来、但别提前给用户看；阶段准入期间两张表短暂不一致是正常的。

02 PRINCIPLE

2.2 整体数据流转架构

写入 → 存储 → 读取：成因在何时写入，读取时又按什么决定返回哪张卡片

整个方案围绕「写入 → 存储 → 读取」三条链路展开。成因在事件发生时写入、或在构建结束时解析； 存储分为终态详情（落库）和运行状态（历史表）；读取时按记录状态决定返回哪张卡片。



图 2 整体数据流转架构：成因在「发生时」或「结束时」写入，读取时按状态决定返回哪种卡片

层次	承载什么	关键点
写入层	取消 / 超时 / 心跳失联 / 暂停终止；构建结束时的统一解析	成因「当场可知」就当场写（取消人、在途插件、失联容器）；「要等结果」的留给构建结束统一对齐
存储层	`构建记录 modelVar`：终态详情 `buildEndInfo`（落库） `构建历史 status`：排队还是运行（实时）	终态落库、运行态不落库。运行态若落库读到的必然是过期快照，还会造成严重写放大
读取层	接口 `getBuildRecordByExecuteCount` → 详情页一张卡片	终态为空才算运行态，保证两张卡片**互斥**；前端「哪个非空渲染哪个」，不要两张叠着画

两套卡片互斥、字段语义不同。终态 `buildEndInfo` 描述「为什么结束、跳到哪」， 覆盖成功 / 失败 / 取消 / 排队超时 / 阶段准入审核中；运行态 `buildRunningInfo` 描述 「此刻卡在哪、在等谁」，覆盖排队中与执行中。两者**不共用**同一套位置结构。

03 DECISION FLOW

三、卡片决策逻辑

打开详情时到底出哪张卡片

用户打开构建详情的那一刻，后端需要根据记录表状态决定返回哪张卡片。 核心先判「运行态」，再判「终态是否需要校正」——记录表状态是唯一判断依据。

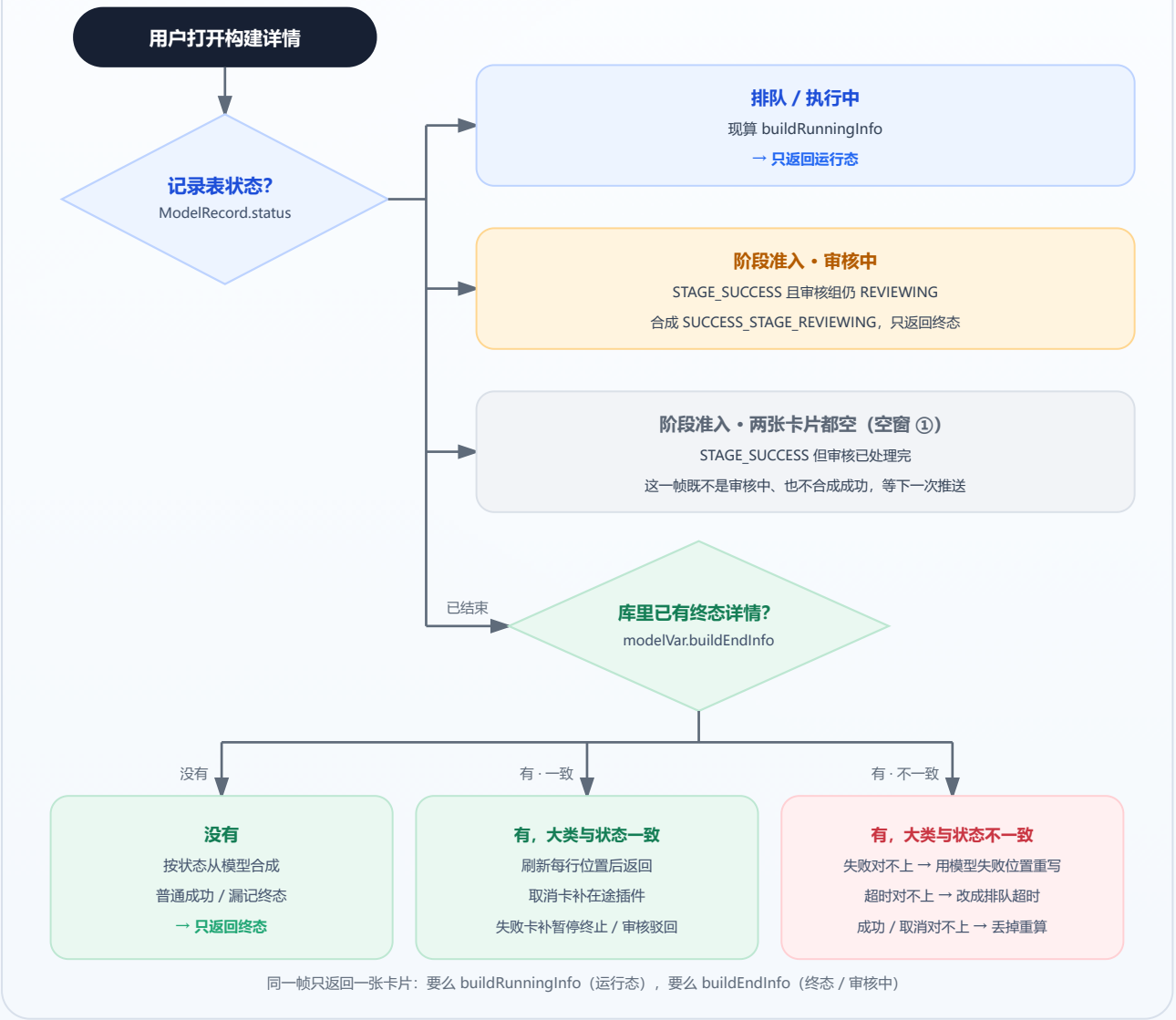


图 3 卡片决策流程：以记录表状态为唯一判断依据，先判运行态，再判终态是否需要校正

03 DECISION FLOW

3.1 页面状态与返回卡片对照表

四种页面形态，对应四种判定路径

用户看到的页面	记录状态	返回卡片
在排队、在执行	`QUEUE` / `QUEUE_CACHE` / 运行中	`buildRunningInfo`
阶段准入等人审核	`STAGE_SUCCESS`，且审核组仍是 `REVIEWING`	`buildEndInfo`，类型 `SUCCESS_STAGE_REVIEWING`
成功、失败、取消、排队超时	对应终态	`buildEndInfo`
最后一组刚点同意、历史还没提交	记录已是运行，或仍是阶段挂起；**历史仍是阶段挂起**	这一帧**可能两张都空**，见第十章空窗①

三个容易忽略的点

记录表是锚点

不用历史表判终态。构建结束的瞬间，记录已经是终态，历史表可能还显示运行中——若用历史表，这一下推送会算不出终态卡片。

「有」不等于直接用

库里已有终态详情时，还要校验**大类是否与记录状态同类**。一致才刷新位置后返回，不一致要覆盖、重写或丢弃后重算。

两张卡片互斥

组装详情时先看终态是否非空：**终态非空才算终态**，否则才去现算运行态。前端也按「哪个非空渲染哪个」。

实现落点： `PipelineBuildFacadeService` 组装详情时，仅当 `buildRecord.buildEndInfo == null` 才会调用 `buildRunningInfoResolver.resolve(...)` 现算运行态——这就是互斥的唯一开关。

04 ARCHITECTURE

四、模块分工与协作

五个模块各司其职 · 入口事件驱动写入

方案涉及五个模块的协作。事件从各个入口触发，经由 engine 层决定写什么， 由 base 层负责落库和读取，process 层做最终组装，前端只负责渲染。

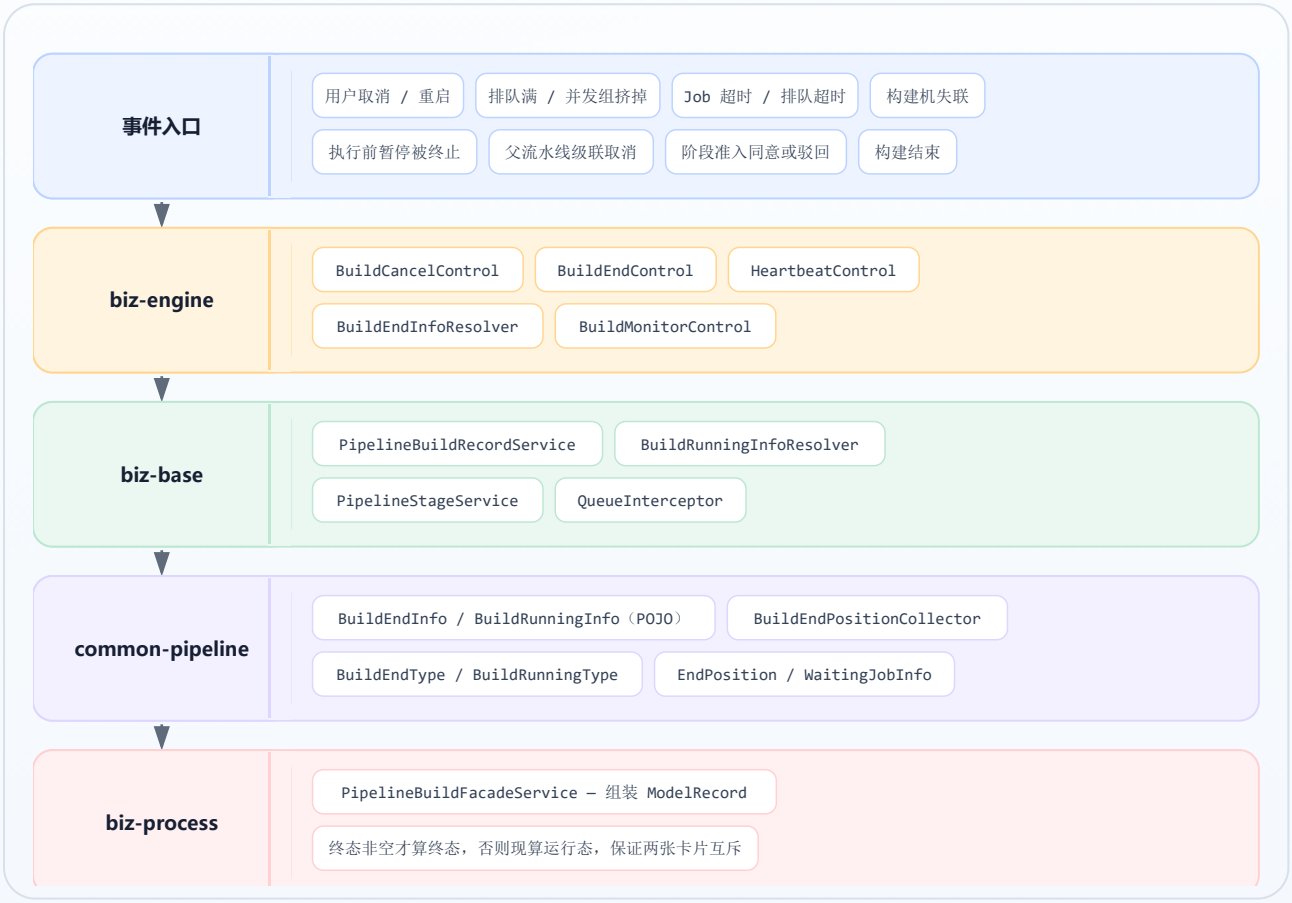


图 4 模块分工架构：事件入口 → engine 决策 → base 读写 → 公共模型 → process 组装

04 ARCHITECTURE

4.1 各模块职责

谁负责写、谁负责读、谁负责组装

模块	负责内容
`common-pipeline`	卡片数据结构定义；从编排模型采集失败 / 取消位置；读取时重算失败类型
`biz-engine`	取消、超时、心跳、构建结束时决定写什么内容
`biz-base`	把卡片写入记录、按状态读出来；现算运行态；阶段准入改记录并推送
`biz-process`	组装详情。终态为空才算运行态，保证两张卡片互斥
`support-files/i18n`	`buildEndType.*` 、`buildRunningType.*` 和各条 `reasonCode` 的多语言文案（zh_CN / en_US / ja_JP）

4.2 核心类索引（写入侧 → 读取侧）

类 / 对象	所属模块	职责
`BuildCancelControl`	biz-engine	处理取消事件：收集在途位置、精炼用户取消原因、区分「普通取消」与「强制终止」
`BuildEndControl`	biz-engine	构建结束入口：调用 Resolver 解析终态，再按兼容性写入或清理
`BuildEndInfoResolver`	biz-engine	终态解析核心：成功 / 异常 / 聚合失败类型 / 与构建状态对齐，以及各原因的回填
`PipelineBuildRecordService`	biz-base	记录读写总入口：解析 / 保存 / 清理终态详情，取消落库，阶段准入审核中合成
`BuildRunningInfoResolver`	biz-base	运行态实时计算：排队位次与并发组、等待 Job、待人工处理项、时长快照
`BuildEndPositionCollector`	common-pipeline	从编排模型采集失败 / 取消位置，聚合构建级失败类型，压缩与截断展示原因
`PipelineBuildFacadeService`	biz-process	组装 `ModelRecord` ；终态与运行态互斥的判定点就在这里

其余入口类（心跳失联、Job / 排队超时、排队挤掉）见附录 A 的完整索引。

05 WRITE PATH

五、终态写入机制

什么时候写 · 谁来写 · 构建结束后如何对齐

5.1 写入时序

终态详情的写入分两个时机：

- **成因已明确时提前写入**：如用户取消、Job 超时、构建机失联。
- **构建结束时统一对齐**：失败、阶段驳回、排队超时。普通成功**不写库**，打开详情时合成。

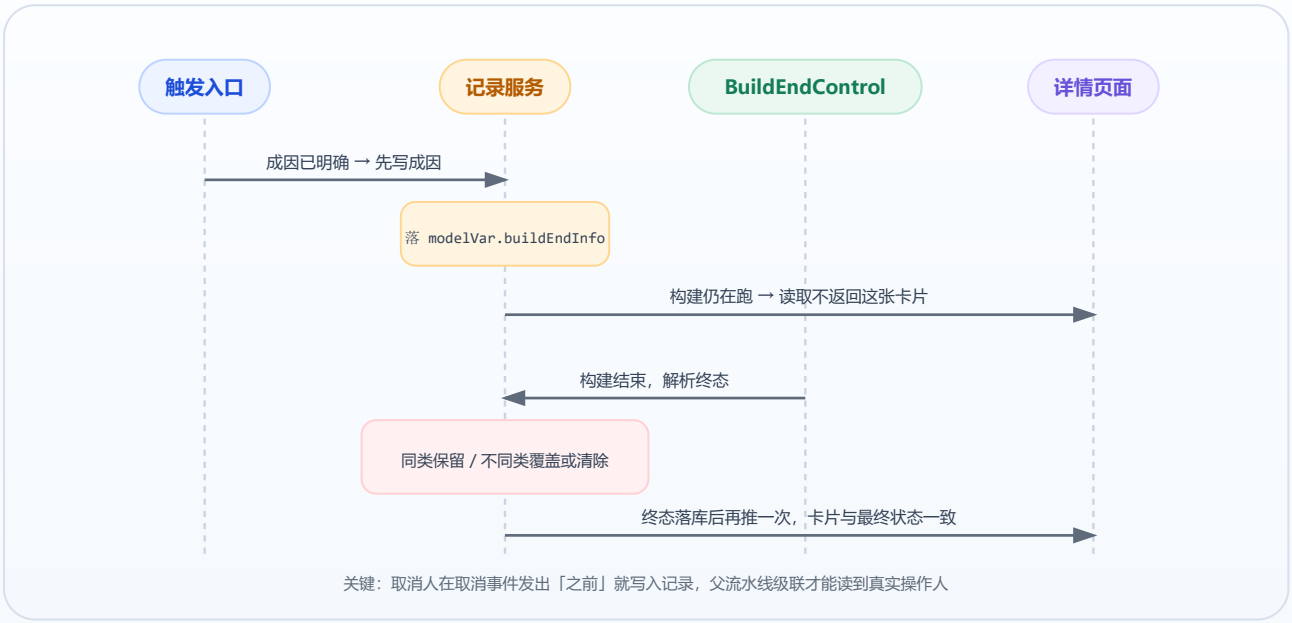


图 5 终态写入时序：提前写成因 → 构建结束时对齐 → 再推一次保证卡片一致

关键设计：取消人在发出取消事件「之前」就写入记录。 因此父流水线级联能读到真实操作人。没有真实操作人时，记录更新会写入字面量 `System`，读取时当成系统取消，**不会展示成某个用户取消的。**

05 WRITE PATH

5.2 全场景写入对照表

谁写入 · 当时写下什么 · 构建结束后页面看到什么

场景	谁写入	当时写下的类型	构建结束后页面看到
用户点取消	详情取消入口 → `BuildCancelControl`	`CANCEL_USER` · 手动取消	取消。收集到在途插件后，文案改成「N 处在途被终止」
60 秒后再点一次，强制停掉	同上，`terminateFlag = true`	`CANCEL_USER` · 强制终止	仍是强制终止，不改成在途计数
重启前先取消旧的那次	重启入口	`CANCEL_USER` · 因重启取消	取消
执行前暂停点被终止	`PipelineTaskPauseListener`，已有卡片则不覆盖	`CANCEL_USER`	构建以取消结束就保留；后来变成失败或成功，则按最终状态改掉
排队满、并发组挤掉、并发优先级挤掉	`QueueInterceptor`、运行时服务	`CANCEL_SYSTEM`	系统取消，原因写明是哪一种
父流水线取消，连带取消子构建	任务服务	`CANCEL_PARENT_PIPELINE`	父流水线取消。父构建先同步写下取消人，子构建再读，卡片上有父流水线名、构建号和操作人
某个 Job 执行超时	监控，并入已有取消卡片	`CANCEL_SYSTEM` + 超时分钟数	跑时不展示；结束后若失败则改失败卡，超时原因保留，位置仍标 Job 超时

怎么读这张表。「当时写下的类型」是写入侧的快照，未必等于用户最终看到的。 用户看到的结果由**构建结束时的那次对齐**决定——所以两列要分开看。

5.2 全场景写入对照表（续）

超时、失败、成功与「不改状态只补原因」的入口

场景	谁写入	当时写下的类型	构建结束后页面看到
构建机失联	心跳，已有卡片则不覆盖	`CANCEL_SYSTEM`（失联）	与 Job 超时相同：跑的时候不展示，结束后按最终状态对齐
在排队队列里等超时	监控	`TIMEOUT_QUEUE`	超时卡片，这是**唯一的构建级超时**
插件失败 / 质量红线 / 人工审核驳回 / 子流水线失败 / 步骤超时	构建结束时解析	按第六章汇总	失败卡片，原因在每一行位置上
阶段准入被驳回，finally 跑完且整体算成功	构建结束时解析	`SUCCESS_STAGE_ABORT`	成功卡片，原因是驳回意见。只有一条意见时直接展示，多条用兜底文案
普通成功	**不写**	无	读取时合成「执行成功」，没有位置列表
服务关闭、删除流水线、编排不存在	各入口先写系统取消	`CANCEL_SYSTEM`	不改这些入口原来的构建状态。最终状态若不是取消，结束时改成与状态同类，关机或删除的原因留在 `reason`

后三行是同一条规则。凡「状态不是这次要改的」入口，都只**补原因**、不改状态；而 `buildEnd` 写最终状态时不受「已取消之后不能再 update」限制。

5.3 几个容易踩坑的细节

这几条都是「不这么写就会出错」的地方

① 运行中重试要清掉上一轮

重试一开始就清掉本轮已写下的终态，避免上一轮的失联或 Job 超时挡住这次结束时的写入。

否则：重试后的构建会带着上一轮的失败卡片。

② Job 超时连续打到多个 Job

后一次把模型里仍处于取消 / 暂停的插件**并进已有卡片**，不把已失败的改回取消，也不覆盖用户取消。

否则：第二个 Job 超时会把用户取消刷成系统取消。

③ 插件被强制终止但自己没有错误码

用事件里的终止原因替换引擎的占位错误码；插件已经报了自己的错误码就保留插件的。

否则：被失败即停连带停掉的人工审核，会被说成「审核驳回」。

④ 子流水线启动时先记名字

子流水线启动时把名称和构建号一并记下。旧数据没有名称时，失败行用兜底文案。 构建号**不写进原因句子**，前端用子流水线信息做跳转。

否则：跨项目跳转会 404。

这三条的共同模式。写入侧面对的是「成因可能被多次追加」的现实，所以每条写入都必须是 **幂等且不降级**的：可以补细节，不能把更准确的信息覆盖成更模糊的。

06 FAILURE RULE

六、失败类型判定规则

构建级失败类型怎么定 · FastKill 与多类失败

每一行位置先按插件自己的终态归类：执行失败、质量红线、人工审核驳回、子流水线、步骤超时、Job 超时。 失败即停（FastKill，错误码 `2199010`）只在写入时能看到终止原因的那些行上标出来， 从编排模型现扫时**标不出这一类。**

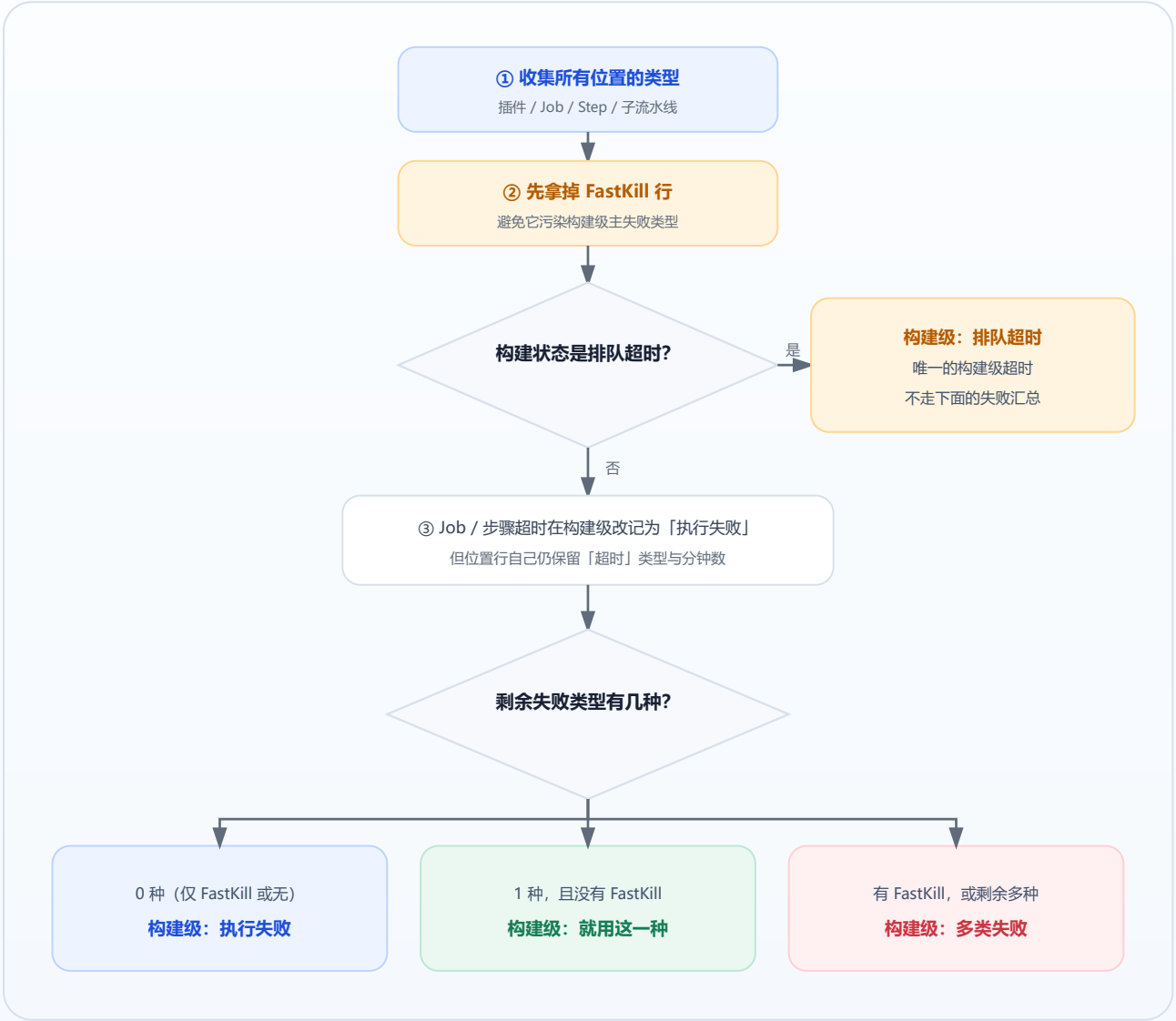


图 6 失败类型判定：FastKill 是触发「多类失败」的关键开关，而不是根因数量统计

06 FAILURE RULE

6.1 判定举例

为什么需要「多类失败」这个大类

为什么需要「多类失败」？ 如果 FastKill 那一行跟构建级用同一种单一失败， 页面会把整行画成错误码 `2199010`，设计稿里的「因失败即停被终止」就出不来。

所以只要 **FastKill 和别的失败同时存在，构建级就写成「多类失败」**—— 这是**页面开关**，不是说有很多个互不相关的根因。

位置上有什么	构建级类型
只有一个插件执行失败	**执行失败**
只有步骤超时，构建状态是失败	**执行失败**（该行仍写「步骤超时」+ 分钟数）
步骤超时 + Job 超时	**执行失败**（两行位置各自保留超时类型）
一个插件失败，同阶段其它 Job 被失败即停终止	**多类失败**（被终止行写「因失败即停被终止」）
只有失败即停、没有别的失败行	**执行失败**
红线未达标 + 插件失败	**多类失败**
一个位置都收集不到（如互斥锁没抢到）	**执行失败**（仍给一张卡，否则详情打不开）
构建状态本身是排队超时	**排队超时**（不走上面的失败汇总）

6.2 原因归谁：构建级 vs 位置级

失败卡片的构建级原因留空

因为同一次构建可能有多处失败，取其中一条会和位置列表打架。

例外：排队超时的原因对整次构建只有一份，留在构建级。

细节都写在该行位置上

每行自己的驳回意见、红线指标、子流水线名、超时分钟数，都写在该行上。

展示时卡片原因压缩空白后截到 **200 字**，插件原始错误只放 tooltip。

容量边界：位置最多 **50 条**，单条原因最长 **512 字**， 展示截断到 **200 字**（超长补「…」），插件原始错误上限 **512 字**。

07 READ PATH

七、读取与动态合成

打开详情时怎么读 · 落库与现算如何对齐

状态以记录表为准。构建结束的瞬间，记录已经是终态，历史表可能还显示运行中—— 若用历史表，这一下推送会算不出终态卡片。读取时的完整决策流程如下：



图 7 读取决策流程：记录表状态是锚点，落库卡片必须与状态大类对齐，否则就地校正

07 READ PATH

7.1 读取时的关键规则

三类校正 · 国际化 · 重算规则的一致性

取消合成：看取消人是谁

取消人有值且不是 `System`，就是**用户取消**；否则是**系统取消**。

用户取消文案里的「N 处」要与卡片上的位置条数对齐。

执行前暂停被终止的那一行，状态文案用「暂停 / 终止」，**不用笼统的「取消」**。

国际化：库里存码，不存句子

`reasonCode` 在读的时候按请求语言翻译。

库里存的是**码和参数**，不是某一门语言的句子——这样切语言不需要重写数据。

三类「大类对不上」分别怎么处理

对不上的情况	处理方式	为什么
失败对不上	用模型里的失败位置**重写**失败卡片；系统取消和父流水线的原因可以留下	失败位置只有编排模型最全，落库的可能不全
排队超时对不上	改成**排队超时**	构建状态本身是 QUEUE_TIMEOUT，卡片必须同类
成功或取消对不上	**丢掉**这份落库，按状态重新合成	这两类没有位置细节可捞，重算最干净

读取重算和写入用同一套规则，出现在三个地方：老构建没有落库、取消卡片被按失败重写、 失败卡片又补上了暂停或审核位置。

模型现扫**补不进 FastKill**；已落库的位置里如果带着它，重算时仍然会得到「多类失败」。

08 STAGE REVIEW

八、阶段准入特殊处理

最容易看错的一段 · 记录与历史短暂不一致

阶段准入是这张卡片里最容易看错的一段，因为记录表和历史表会短暂不一致。 审核挂起时两边都是 `STAGE_SUCCESS`，卡片是「审核中」； 最后一组点同意后，记录先变运行，历史表要等事务提交才变运行。



图 8 阶段准入时序：同意与驳回三条分支，记录表与历史表会短暂不一致

08 STAGE REVIEW

8.1 关键规则

挂起 · 驳回 · 真正结束三种情形的不同处理

挂起时：不走运行态

- 位置**从 1 开始编**，触发器阶段不算。
- 卡片上有审核组序号、组名、待审核人和已等待时长（`waitCostTime`）。
- 合成入口：`PipelineBuildRecordService` 中按 `stage.checkIn / checkOut` 里 `status == REVIEWING` 定位审核阶段。

驳回时：不能把记录改成运行

> 记录一旦进入已取消，后续**普通更新改不动这个状态**，finally 期间详情才能稳定停在已取消。
>
> 历史表改回运行是**原有调度需要**（为了把 finally 跑起来），和详情卡片用的**不是同一张表**。

真正结束时：buildEnd 直接写最终状态

- `buildEnd` 不受「已取消之后不能再 update」这条限制。
- 若阶段上有驳回记录且整体成功，写入 `SUCCESS_STAGE_ABORT`。

阶段	记录表	历史表	页面卡片
审核挂起	`STAGE_SUCCESS`	`STAGE_SUCCESS`	审核中
同意 · 还有下一组	仍挂起，下一组 `REVIEWING`	仍 `STAGE_SUCCESS`	继续审核中
同意 · 最后一组	先阶段成功 → 改为运行	事务提交后改回运行	空一帧 → 执行中
驳回	**保持已取消（System）**	改回运行（调度 finally）	系统取消 → 阶段准入驳回

09 RUNNING STATE

九、运行态实时计算

排队和执行中现算什么 · 不落库的原因

运行态只看历史表。排队是 `QUEUE` / `QUEUE_CACHE`， 执行中是运行中的那些状态。阶段挂起、触发待审核、已经结束，这里直接返回空。 **计算抛错也返回空**——详情页仍然打得开。

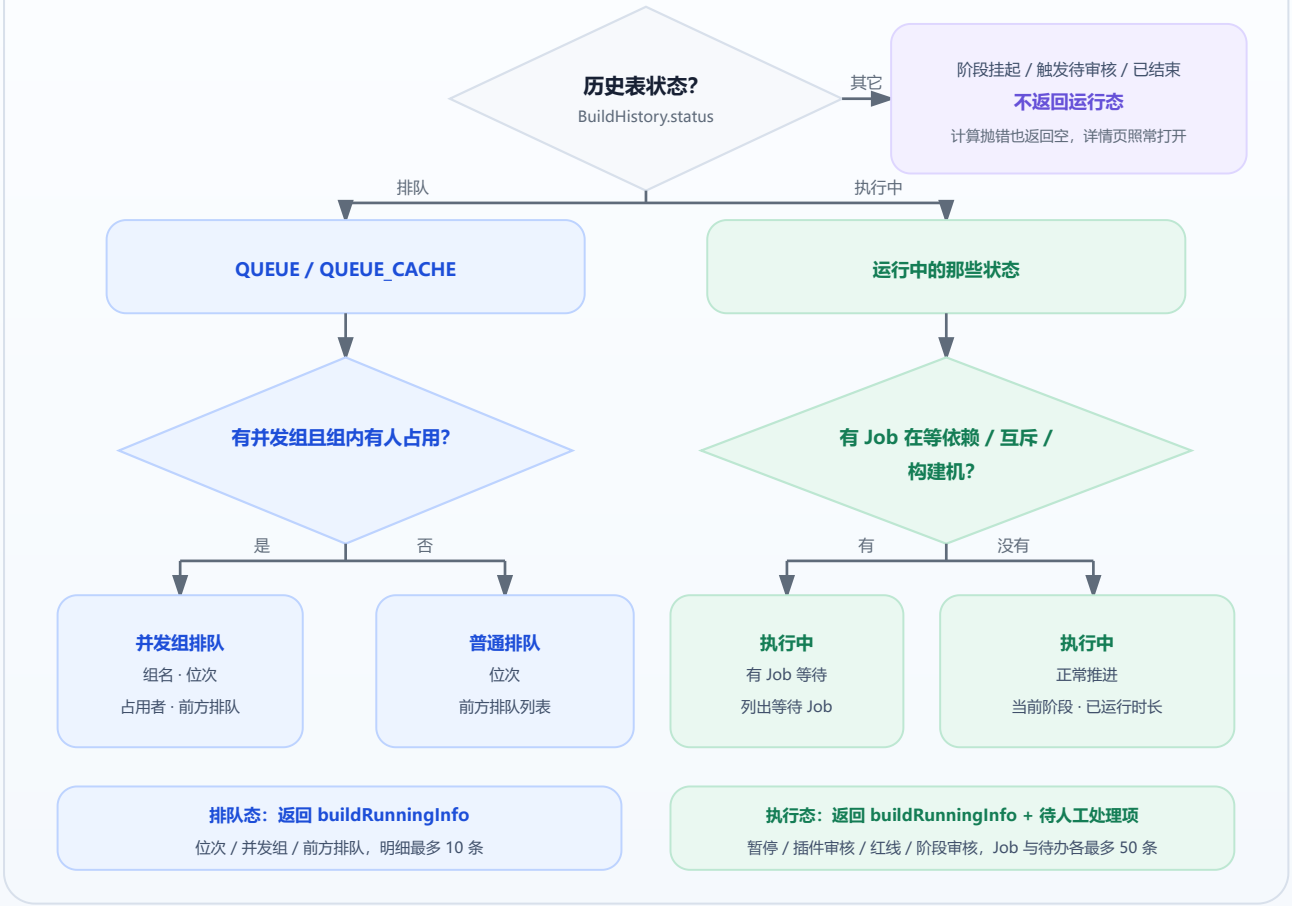


图 9 运行态计算流程：排队分「并发组 / 普通」，执行中分「有 Job 等待 / 正常推进」

为什么运行态不落库? 排队位置、前方排队列表、已等待时长每秒都在变， 落库读到的必然是**过期快照**，还会造成严重写放大。

所以只在「读取详情且构建未结束」时现算——**对已结束的构建零额外开销。**

09 RUNNING STATE

9.1 / 9.2 排队与执行中的计算规则

位次怎么算 · 什么时候算「在等资源」· 时长谁来加

9.1 排队计算规则

项目	规则
查询范围	按并发组或当前流水线查，只取展示用的列；正式构建和**未删除的调试构建**合在一起
排序与位次	用入队时间排序，同一毫秒再看构建号；位次从 **1** 开始
找不到自己	如并发组配置刚改过，位次给 **0**，前端不显示「第几位」
明细与上限	明细最多 **10** 条，多出来的只计数；并发上限与流水线设置一致，单条流水线锁和并发组锁都是 **1**

9.2 执行中计算规则

项目	规则
资源排队宽限	准备环境的前 **15 秒**视为构建机正常拉起，不报「在等资源」；构建机复用互斥按复用配置**直接**给出等待原因，不必等满 15 秒
标题不变	等人处理的项目**不改卡片标题**，也不改当前阶段名
列表上限	等待的 Job 和待处理项最多各 **50** 条
时长口径	时长是服务端算出快照的那一刻。前端用详情上的 `currentTimestamp` 自己每秒加

> **不要为刷新秒数高频轮询。**服务端只给一次时长快照 + 一个时间基准，前端本地自增即可。

9.3 运行态子类型全集

大类	枚举值	中文标签 / 含义
QUEUE	`QUEUE_WAITING`	排队中（流水线自身串行 / 并发额度未释放）
	`QUEUE_CONCURRENCY`	并发组排队中（被其它构建占用）
RUNNING	`RUNNING_NORMAL`	执行中（正常推进）
	`RUNNING_JOB_WAITING`	执行中（有 Job 因互斥 / 资源 / 依赖在等待）

10 PUSH

十、推送机制与空窗说明

什么时候推给页面 · 为什么会有空窗

10.1 推送时机

记录变更**等事务提交后再推**，避免推送读到还没提交的状态。终态详情若在记录状态更新之后才写入， **会再推一次**——否则用户要手动刷新才看得到卡片。

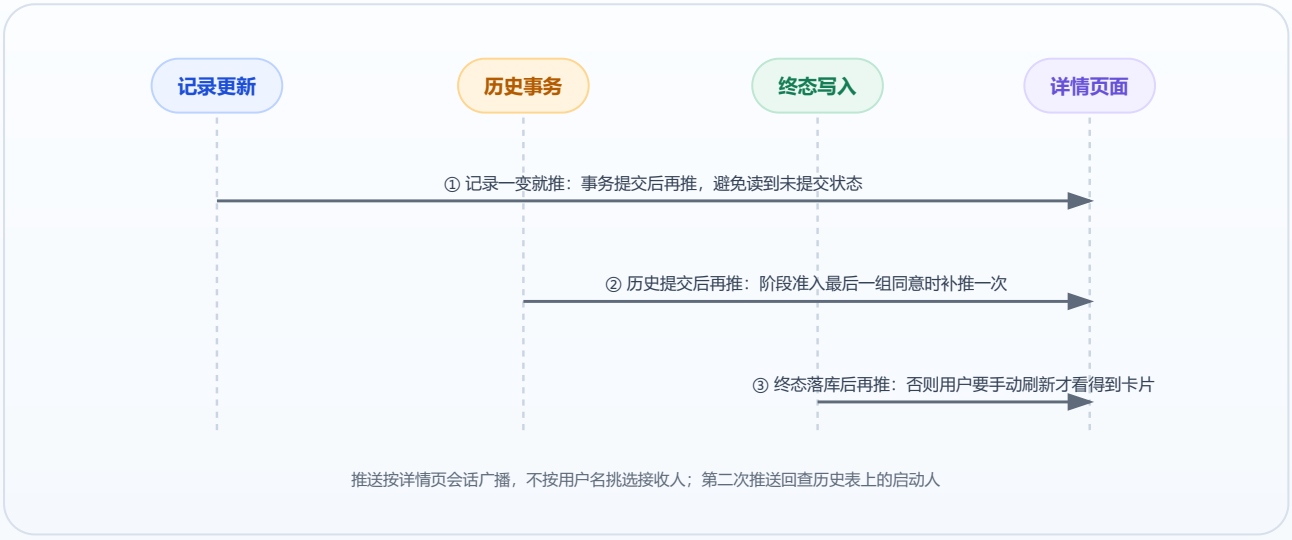


图 10 三次推送时机：记录变更 → 历史提交 → 终态落库，保证卡片不会晚一帧

推给谁。推送按**详情页的会话广播**，不按某一个人的用户名挑选接收人。事件上带的是构建启动人。

阶段准入最后一组同意时，调用方手里的是**点同意的审核人**（不是启动人）， 所以第二次推送不拿审核人去填，而是**回查历史表上的启动人**。

10.2 三段已知空窗

只影响中间某一帧 · QA 按这个预期看，不是 bug

结论先行：最终落库是对的。这三段只影响中间某一帧的展示，都属于推送时序导致的正常空窗。

空窗 ①

最后一组点同意后

审核结果刚落库的那一次推送会**空一帧**。

记录还是阶段挂起，历史还没改回运行，模型里已经没有「审核中」—— 既不是审核中，也不是执行成功。

随后：历史提交后的推送变成「执行中」。

空窗 ②

驳回之后、finally 跑完之前

详情是「已取消 / 系统取消」，历史列表可能已经显示运行中。

随后：构建结束后才变成「阶段准入驳回」。

空窗 ③

驳回并以阶段成功收尾

构建结束会先推一次。这时终态还没写入，历史仍是运行中，可能先闪一张运行中卡片。

随后：同一次结束流程里的下一次推送才是阶段准入驳回。

空窗对照：一帧之后会变成什么

空窗	这一帧用户可能看到	下一帧
①	两张卡片都空	执行中（`RUNNING_NORMAL`）
②	已取消 / 系统取消（历史列表可能显示运行中）	阶段准入驳回（`SUCCESS_STAGE_ABORT`）
③	先闪一张运行中卡片	阶段准入驳回

给 QA 的提醒：这三段不要报「状态写错」。判断标准是 **最终落库的大类与页面左上角状态一致**，而不是中间某一帧是否出现过渡态。

11 DATA CONTRACT

十一、数据结构与前端对接

buildEndInfo 与 buildRunningInfo 核心字段

两套卡片结构**不共用**同一套位置 / 关联构建结构。终态的位置描述「为什么结束、跳到哪」； 运行态描述「此刻卡在哪、在等谁」，字段语义不同，不要互相复用。

11.1 终态详情 buildEndInfo

字段	类型	含义 / 前端用法
`endCategory`	String	结束大类： `CANCEL` / `FAIL` / `TIMEOUT` / `SUCCESS` 。决定卡片样式与配色
`endType`	枚举	结束子类型全集，见下表。前端按 category 归类，不硬编码每个子类型
`endTypeDesc`	String	已翻译成因标签，如「系统取消」。卡片内原因标签直接展示
`reasonCode` / `reasonParams`	String / List	国际化码与参数。**库里存码不存句子**，读时按请求语言翻译
`reason`	String?	构建级原因（失败类恒为 null）。卡片头部原因详情，空则不渲染
`operator`	String?	操作人（仅用户主动取消有值）。展示「由 xxx 取消」
`positions`	List?	被影响的组件位置列表（最多 50 条）。渲染受影响位置，支持跳转定位
`parentPipelineInfo`	Object?	父流水线信息（仅级联取消有值）。含项目 ID、流水线名、构建号、操作人，可跳转
`totalCostTime`	Long?	构建总耗时（毫秒）。展示「已运行 N 分钟」
`waitCostTime`	Long?	已等待时长，仅 `SUCCESS_STAGE_REVIEWING` 有值
`endTime`	Long?	终态时间戳（毫秒）。按需格式化展示结束时间
`positionCount`	Int	被影响位置总数。展示「N 个位置受影响」用它，不用 `positions.length`

11.2 endType 枚举全集

15 个子类型，按大类分组（构建级 / 位置级已标注）

大类	枚举值	中文标签 / 说明
CANCEL	`CANCEL_USER`	用户取消
	`CANCEL_SYSTEM`	系统取消（含服务重启、排队满、并发互斥、构建机失联等）
	`CANCEL_PARENT_PIPELINE`	父流水线取消
FAIL	`FAIL_EXEC`	执行失败（插件执行失败等常规失败）
	`FAIL_QUALITY`	质量红线未达标
	`FAIL_REVIEW`	人工审核驳回
	`FAIL_SUB_PIPELINE`	子流水线执行失败导致父构建失败
	`FAIL_FAST_KILL`	因失败即停被终止 **仅位置级**：连带影响，非独立根因
	`FAIL_MULTIPLE`	多类失败（同时存在多种失败子类型）
TIMEOUT	`TIMEOUT_QUEUE`	**排队超时** —— 唯一的构建级超时（构建状态本身即 QUEUE_TIMEOUT）
	`TIMEOUT_JOB`	Job 超时 **仅位置级**：构建级归取消 / 失败类
	`TIMEOUT_STEP`	步骤超时 **仅位置级**：构建级归失败类
SUCCESS	`SUCCESS`	执行成功
	`SUCCESS_STAGE_ABORT`	已驳回（阶段准入驳回，按阶段成功结束）
	`SUCCESS_STAGE_REVIEWING`	审核中（阶段准入挂起，通过后继续运行）

三条容易搞混的边界。

① **构建级只有 TIMEOUT_QUEUE 一个超时**；Job / 步骤超时是位置级类型，构建级会归到取消或失败。

② **FAIL_FAST_KILL 只是位置级**，表示「被别的失败连带终止」，不是独立根因。

③ **SUCCESS_STAGE_ABORT 归 SUCCESS 大类**：驳回后构建按成功结束，所以是成功卡片 + 驳回原因。

11.3 运行态详情 buildRunningInfo

字段语义与前端用法

字段	类型	含义 / 前端用法
`runningCategory`	String	运行大类：`QUEUE` / `RUNNING`。决定卡片样式（排队 / 执行配色）
`runningTypeDesc`	String	已翻译状态标签。用作卡片标题（排队中 / 执行中）
`currentPhase`	String?	当前阶段名（有 Job 排队时为「Job 正在排队」）。执行中卡片的当前阶段
`queueDetail`	Object?	排队详情：位次、并发组、并发上限、占用中构建、前方排队。排队态填充
`waitingJobs`	List?	等待中的 Job 列表（最多 50 条），含等待类型（互斥 / 资源 / 依赖）。执行态填充
`pendingItems`	List?	待人工处理项（暂停 / 插件审核 / 红线 / 阶段审核，最多 50 条）。执行态附加展示
`waitingTime` / `runningTime`	Long?	已等待 / 已运行时长（毫秒快照）。结合 `currentTimestamp` 自增展示
`queueDetail.queuePosition`	Int	位次。为 **0** 时不显示「第几位」
`queueTime` / `startTime`	Long?	入队时间 / 开始执行时间（毫秒）。展示「20:01:00 入队」「20:01:00 开始执行」
`triggerUser` / `triggerDesc`	String?	触发人 / 已翻译的触发方式（手动触发、Git 事件触发）。展示「fayeqiang 手动触发」
`*Count` 系列	Int	列表真实长度（含 `waitingJobCount` / `pendingItemCount`）。**列表本身有截断**，计数用它而不是 `list.length`

前端对接三条避坑（必看）。

① **跨流水线跳转一律用对应对象自己的 projectId**（父 / 子 / 依赖 Job），不能用当前页面项目 ID，否则跨项目时 **404**。

② **列表长度用 `xxxCount` 字段**，不要用 `list.length` ——后端有截断。

③ **不要为刷新秒数高频轮询**，用 `currentTimestamp` 本地自增。

12 BOUNDARY

十二、明确不改的行为

这些边界刻意保留 · 不要在后续迭代中误改

不改的行为	原因
不在构建结束时扫仍停在 `QUEUE` / `QUEUE_CACHE` 的任务并改引擎状态	那次扫描赶不上取消后已经卡住的 Job，也会和还在认领任务的构建机不一致
单个插件变成取消时，**不删** `CANCEL_ACTION_TIME` 标记	60 秒后第二次取消会不会升级成强制终止，靠这个标记判断
服务关闭、删除流水线、编排不存在，**维持原来的构建状态**，只补原因	这些入口原来的构建状态有其语义，不应被详情卡片改写
`stageCancel` 继续把记录写成已取消	这是取消动作，记录需要停在终态上，finally 才能稳定跑完

为什么单列一章。这四条看起来像「实现得不彻底」或「可以顺手优化」的地方， 但实际上每一条都有明确的时序或一致性问题作为理由。改动它们会引入更难查的 Bug。

相关常量速查

常量	值	用途
`CANCEL_ACTION_TIME_{buildId}`	Redis Key	判断 60 秒内是否二次取消，决定是否升级为强制终止
`USER_STAGE_FASTKILL_TERMINATE`	2199010	失败即停（FastKill）终止的错误码
`POSITION_MAX_SIZE`	50	终态位置列表上限
`REASON_DISPLAY_MAX`	200	卡片展示原因截断长度
`RELATED_BUILD_MAX_SIZE`	10	排队关联构建明细上限
`LIST_MAX_SIZE`	50	等待 Job / 待处理项上限
`RESOURCE_QUEUE_GRACE_MS`	15_000	准备环境的资源排队宽限期

13 RELEASE

十三、上线与验证

发布范围 · 单测覆盖 · QA 验证清单

13.1 发布范围

模块	内容
`common-pipeline`	数据结构与位置采集
`biz-base`	记录读写、运行态现算、阶段准入
`biz-engine`	取消 / 结束时的写入决策
`biz-process`	详情组装
`support-files/i18n`	新增的多语言词条（zh_CN / en_US / ja_JP 三语种）

13.2 单测覆盖

已通过（无数据库单测）： `BuildEndPositionCollectorTest` 、 `BuildEndInfoAlignTest` 、 `BuildEndInfoResolverTest` 。

待 CI： `biz-base` / `biz-engine` 的 jOOQ 生成需要 MySQL，本机未编，**以 CI 编译为准。**

13.3 QA 验证清单

#	验证场景
1	用户取消、60 秒后强制终止、重启前取消、父流水线连带取消、排队超时——卡片大类和左上角状态一致，父流水线卡片可跳转
2	失败构建里只有步骤超时或 Job 超时——构建级是执行失败，该行仍是超时；跑时不出现取消卡片
3	一个插件失败 + 同阶段开启失败即停——卡片是多类失败，被终止行是「因失败即停被终止」，不是 2199010
4	两个 Job 先后超时——位置补齐且不盖掉用户取消；暂停被终止的文案是「暂停 / 终止」
5	阶段准入——等人时「审核中」；最后一组同意不出现「执行成功」，空一帧后变执行中；驳回后保持已取消，成功结束后是阶段准入驳回
6	排队能看到第几位、谁占用、前面有谁；找不到自己不显示位次；等待 Job 不改卡片标题；准备环境 15 秒内不报资源排队
7	运行中重试之后，上一轮的失联或 Job 超时卡片不再出现

APPENDIX A

附录 A　代码与常量索引

模块根目录：`src/backend/ci` · 便于按图索骥

A.1 数据结构（common-pipeline）

文件	说明
…/pojo/BuildEndInfo.kt …/pojo/BuildRunningInfo.kt	终态主对象（`MODEL_VAR_KEY = "buildEndInfo"` 落库 key）；运行态主对象（含 `BuildQueueDetail`）
…/pojo/EndPosition.kt …/pojo/WaitingJobInfo.kt …/pojo/PendingManualItem.kt	三类位置与关联结构：子流水线、依赖 Job、等待 Job、待人工处理项
…/enums/BuildEndType.kt	15 个终态子类型 + `BuildEndCategory`
…/enums/BuildRunningType.kt	4 个运行态子类型 + `BuildRunningCategory`
…/utils/BuildEndPositionCollector.kt	位置采集、失败类型聚合、原因压缩与截断

A.2 写入侧（biz-engine）

文件	说明
core/process/biz-engine/src/main/kotlin/com/tencent/devops/process/engine/control/BuildEndControl.kt	构建结束入口：`saveBuildEndInfo() → resolve → saveIfCompatible / clearIfIncompatible`，整体 try/catch 兜底
…/control/BuildCancelControl.kt	取消事件：收集位置、精炼用户取消原因、按 `TERMINATE` 分流
…/control/BuildEndInfoResolver.kt	终态解析：成功 / 异常 / 聚合类型 / 与状态对齐，及各原因回填
…/control/HeartbeatControl.kt …/control/BuildMonitorControl.kt	构建机心跳失联；Job 执行超时、排队队列超时
core/process/biz-base/…/engine/interceptor/QueueInterceptor.kt	排队满 / 并发组挤掉 → `ofCancelSystem`

APPENDIX A

附录 A（续）读取侧与单测索引

模块根目录：`src/backend/ci` · 便于按图索骥

A.3 读取侧（biz-base / biz-process）

文件	说明
core/process/biz-base/…/engine/service/record/PipelineBuildRecordService.kt	终态解析 / 保存 / 清理，取消落库，阶段准入「审核中」合成
core/process/biz-base/…/engine/service/record/BuildRunningInfoResolver.kt	运行态现算：排队位次与并发组、等待 Job、待人工处理项、时长快照
core/process/biz-process/…/service/builds/PipelineBuildFacadeService.kt	详情组装；`buildEndInfo == null` 时才现算运行态（互斥开关）
core/process/api-process/…/api/user/UserBuildResource.kt	接口 `getBuildRecordByExecuteCount` 声明
core/process/biz-process/…/api/UserBuildResourceImpl.kt	接口实现

A.4 单测

文件	说明
core/common/common-pipeline/src/test/kotlin/…/utils/BuildEndPositionCollectorTest.kt	位置采集与失败聚合
core/common/common-pipeline/src/test/kotlin/…/pojo/BuildEndInfoAlignTest.kt	终态与状态对齐
core/process/biz-engine/src/test/kotlin/…/control/BuildEndInfoResolverTest.kt	终态解析

— 文档结束 —

蓝盾团队 · CI/CD 技术中台 · 构建详情状态小结方案 v1.3